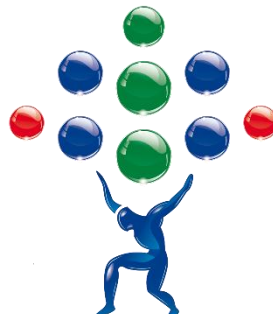


UNIVERSIDAD PRIVADA DOMINGO SAVIO

Ingeniería de Sistemas



UNIVERSIDAD PRIVADA
DOMINGO SAVIO

TRABAJO:

Actividad 6: "Sprint Ready – El Gran Reset de Ingeniería"

ESTUDIANTE (S):

Josue Adhemar Mamani Huanacu

Josue Matias Arroyo Reynoso

Luis Fernando Arias Segovia

DOCENTE:

Ing. Nelson Huanca Victoria

Profesionales + humanos

Sistema DSS de Ventas de Repuestos Automotrices y Facturación Electrónica

1. Enlace Github

ETAPA A: Diagnóstico y Propuesta de Valor

2. Contexto Estratégico

2.1 Resumen del problema (Árbol de problemas)

El siguiente árbol de problemas identifica la causa raíz que justifica el desarrollo del sistema como un DSS (Decision Support System) y no como un software transaccional convencional.

El tronco (Problema central):

Los negocios minoristas no cuentan con visibilidad financiera en tiempo real sobre la rentabilidad real de su inventario, ya que los datos de venta y de facturación se generan y almacenan de forma aislada, sin un mecanismo que los transforme en información analítica para la toma de decisiones.

Las raíces (Causas):

- Los registros de venta y los comprobantes fiscales se gestionan en sistemas o medios independientes (hojas de cálculo, talonarios o aplicaciones de facturación aisladas), sin integración entre sí.
- No existe un mecanismo automatizado que cruce la tasa de rotación de cada producto con su margen de ganancia neta.
- Las decisiones de compra, precio y reposición de inventario se basan en la experiencia o intuición del personal administrativo, y no en datos históricos.
- La información generada por las ventas diarias no se consolida ni se actualiza en tiempo real, por lo que los reportes, cuando existen, llegan de forma tardía.
- El proceso de facturación electrónica, al no estar conectado al inventario, no aporta valor más allá del cumplimiento fiscal.

Las ramas y hojas (Efectos):

- Sobre-stock de productos de baja rotación y bajo margen (Productos Hueso), inmovilizando capital de trabajo.

- Desabastecimiento de productos de alta rotación y alto margen (Productos Estrella), generando pérdida de ventas.
- Reducción de la rentabilidad neta del negocio por decisiones de compra mal informadas.
- Incapacidad de reaccionar oportunamente ante cambios en la demanda o en los márgenes de los proveedores.
- Dependencia de procesos manuales que incrementan el riesgo de error humano y de incumplimiento fiscal.

Este árbol de problemas evidencia que la causa raíz no es la ausencia de un sistema de registro de ventas, sino la falta de un mecanismo analítico que conecte los datos transaccionales con la toma de decisiones. Por ello, un sistema puramente CRUD, que se limite a registrar ventas y facturas, no resuelve el problema de fondo; se requiere un Sistema de Soporte de Decisiones (DSS) capaz de procesar los datos de venta y facturación para clasificar el inventario y generar conocimiento accionable a través de un dashboard interactivo.

2.2 Árbol de soluciones (La Estrategia)

El siguiente árbol de soluciones plantea la estrategia que responde directamente al árbol de problemas, transformando cada causa identificada en un medio de acción y cada efecto en un fin esperado.

El tronco (Objetivo central):

Los negocios minoristas cuentan con visibilidad financiera en tiempo real sobre la rentabilidad de su inventario, gracias a un Sistema de Soporte de Decisiones (DSS) que integra los datos transaccionales de venta y facturación electrónica, transformándolos en información analítica accionable a través de un dashboard interactivo.

Las raíces (Medios):

- Integrar el módulo de facturación electrónica con el sistema de ventas e inventario en una única base de datos relacional, eliminando los registros aislados en hojas de cálculo o aplicaciones independientes.
- Implementar un modelo matricial que clasifique automáticamente los productos cruzando la tasa de rotación con el margen de ganancia neta (Producto Estrella vs. Producto Hueso).
- Proveer un dashboard con indicadores visuales que sustente las decisiones de compra y precio en datos reales, reduciendo la dependencia de la intuición del personal administrativo.

- Diseñar el modelo de datos con campos indexados por fecha (timestamps) que permitan la actualización y consulta de métricas del negocio en tiempo real, evitando reportes tardíos.
- Aprovechar la facturación electrónica como mecanismo de ingesta de datos limpios y estandarizados que alimenten directamente el motor analítico del DSS, dándole valor más allá del cumplimiento fiscal.

Las ramas y hojas (Fines):

- Optimización del capital de trabajo mediante la reducción del stock inmovilizado en productos de baja rotación y bajo margen.
- Disponibilidad oportuna de los productos de alta rotación y alto margen, evitando la pérdida de ventas por desabastecimiento.
- Incremento de la rentabilidad neta del negocio gracias a decisiones de compra y precio mejor informadas.
- Mayor capacidad de respuesta ante cambios en la demanda o en los márgenes ofrecidos por los proveedores.
- Reducción del riesgo de error humano y de incumplimiento fiscal mediante la automatización de los procesos.

Este árbol de soluciones demuestra que, al atacar directamente las causas identificadas mediante la integración de la facturación electrónica y un modelo analítico de clasificación, el sistema deja de ser un simple registro transaccional y se convierte en un verdadero DSS, capaz de generar los fines esperados en términos de rentabilidad y eficiencia operativa.

2.3 Objetivo SMART del proyecto

Implementar dentro del plazo establecido en el cronograma de la asignatura, un Sistema de Soporte de Decisiones (DSS) integrado a un módulo de facturación electrónica para negocios minoristas, capaz de clasificar de manera automática el catálogo de productos en categorías de rentabilidad (Producto Estrella y Producto Hueso) mediante el cruce de la tasa de rotación y el margen de ganancia neta, con el fin de incrementar la visibilidad financiera del negocio y apoyar la toma de decisiones de compra, precio y reposición de inventario del equipo administrativo; el cumplimiento de este objetivo se medirá a través de la entrega de un dashboard funcional que procese la totalidad de las transacciones registradas en el sistema y clasifique el 100% del catálogo de productos activos antes de la fecha límite de entrega del proyecto.

3. Definición del MVP

3.1 Cuadro “Es / No es / Hace / No hace”

ES	NO ES
Un Sistema de Soporte de Decisiones (DSS) que transforma los datos transaccionales de venta y facturación en información analítica para la gestión del inventario. Una solución que integra la facturación electrónica como fuente de datos limpios y estandarizados para el análisis. Una herramienta de apoyo a la toma de decisiones de compra, precio y reposición basada en indicadores de rotación y margen de ganancia.	Un sistema CRUD convencional limitado únicamente al registro de ventas y productos. Un sistema contable o un ERP completo que gestione la totalidad de la operación financiera del negocio. Un sistema de facturación electrónica certificado de forma autónoma ante la entidad fiscal correspondiente.
HACE	NO HACE
Registra ventas, detalle de venta y productos, conservando el precio histórico de cada transacción. Calcula automáticamente la tasa de rotación y el margen de ganancia neta por producto. Clasifica el inventario en categorías de rentabilidad (Producto Estrella / Producto Hueso) mediante un modelo matricial. Genera un dashboard interactivo con indicadores y gráficos de apoyo a la decisión. Controla el acceso al módulo DSS mediante roles de usuario (administrador, vendedor).	No gestiona la contabilidad general ni la declaración de impuestos del negocio. No envía ni valida comprobantes directamente ante la entidad fiscal; esa función se asume como un servicio externo o de terceros. No gestiona logística de despacho, rutas de entrega ni control de proveedores. No realiza predicciones de demanda mediante modelos de machine learning en esta fase del proyecto.

3.2 Canvas MVP

Elemento	Descripción
Propósito / Visión	Brindar a los negocios minoristas visibilidad financiera en tiempo real sobre la rentabilidad de su inventario, integrando la facturación electrónica como fuente de datos confiable.

Público objetivo	Propietarios y personal administrativo de negocios minoristas que actualmente gestionan sus ventas y su facturación sin herramientas analíticas integradas.
Problema que resuelve	La falta de visibilidad sobre qué productos son realmente rentables (alta rotación y margen) frente a aquellos que inmovilizan capital de trabajo (baja rotación y margen).
Funcionalidades imprescindibles (Must have)	Registro de ventas y facturación electrónica; cálculo de tasa de rotación y margen de ganancia; clasificación de productos en Estrella/Hueso; dashboard básico; control de acceso por roles.
Funcionalidades deseables (Nice to have)	Alertas automáticas de reposición de inventario; reportes exportables; proyecciones de tendencia de ventas.
Hipótesis a validar	Si el negocio cuenta con un dashboard que clasifique su inventario por rotación y margen, podrá tomar decisiones de compra y precio más acertadas, mejorando su rentabilidad.
Métricas de éxito	Porcentaje del catálogo de productos clasificado correctamente; tiempo de generación del dashboard; reducción del stock inmovilizado de productos de baja rotación.
Principal riesgo	Que la calidad o disponibilidad de los datos provenientes de la facturación electrónica no sea suficiente para alimentar el modelo analítico en tiempo real.

ETAPA B: MODELADO ESTRUCTURAL

4. Diseño Técnico (UML)

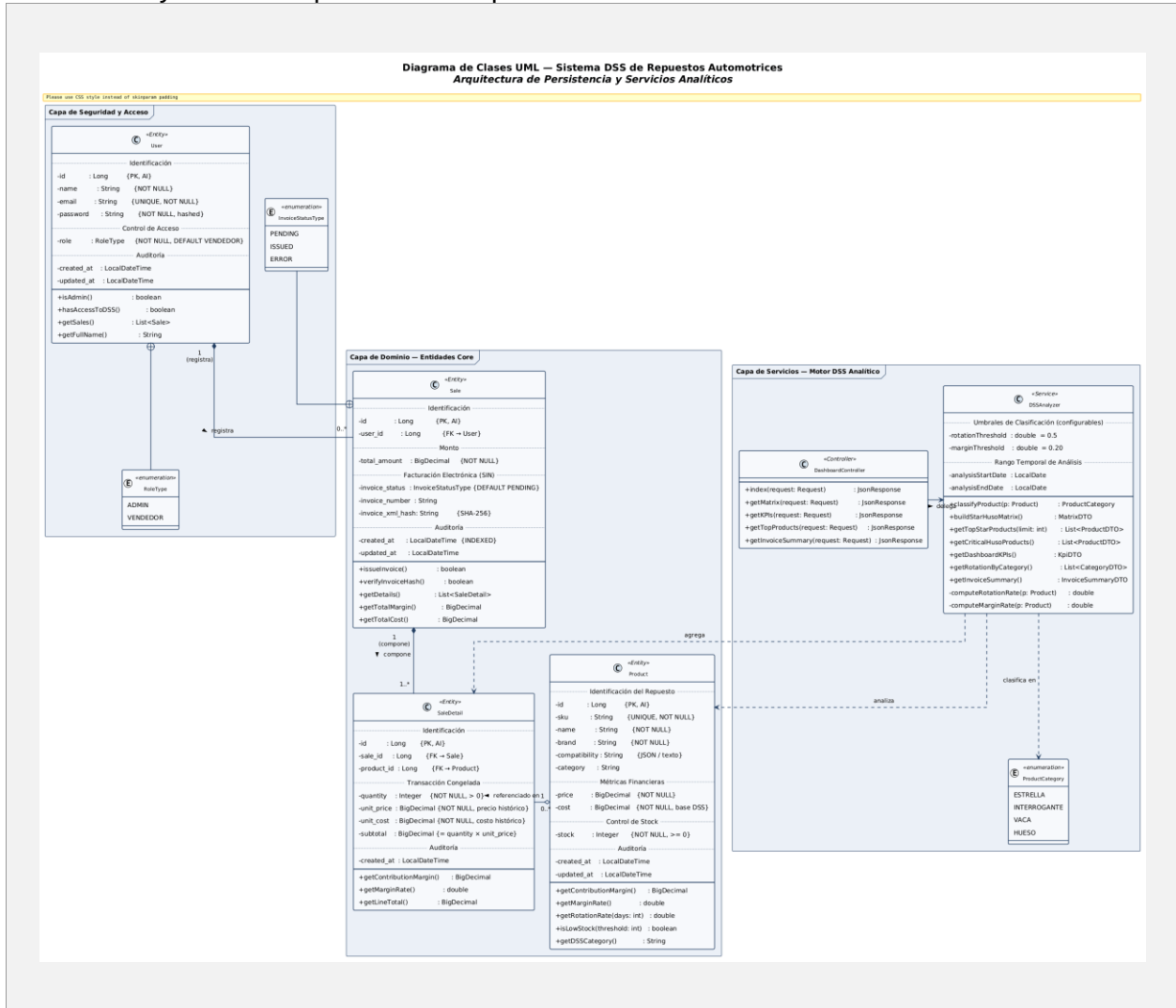
4.1 Diagrama de Clases de Persistencia

El diagrama de clases modela las entidades del dominio con sus atributos, visibilidades y métodos de negocio, así como las relaciones de composición entre ellas. Las clases principales son: User, Product, Sale y SaleDetail.

Relaciones clave:

- User (1) —> Sale (N): Un usuario vendedor puede registrar múltiples ventas. Composición: la venta depende del usuario.
- Sale (1) —> SaleDetail (N): Cada venta se compone de uno o más detalles. Composición fuerte.
- Product (1) —> SaleDetail (N): Un producto puede aparecer en múltiples detalles de venta.

- DSSAnalyzer (servicio): Clase de servicio que consume las agregaciones sobre Sale y SaleDetail para clasificar productos en Estrella/Hueso.



4.2 Diagrama de Secuencia de la Lógica de Decisión

El diagrama de secuencia modela el flujo más complejo del sistema: la carga del Dashboard DSS. Este flujo inicia cuando el usuario administrador solicita el panel analítico, recorre la cadena React → Laravel API → Base de Datos → Motor DSS → Respuesta JSON → Renderizado de gráficos.

Actores y participantes en la secuencia:

- Admin (Usuario): Rol con acceso al módulo Dashboard.
- DashboardPage (React): Componente frontend que monta el dashboard al inicializarse.
- AxiosClient (React): Capa de servicio HTTP que encapsula las llamadas a la API.
- DashboardController (Laravel): Controlador API que orquesta la lógica analítica.
- DSSAnalyzer (Laravel Service): Servicio que ejecuta las consultas matriciales sobre la BD.

- Database (MySQL): Motor de persistencia que responde a las consultas agregadas.

Please use CSS style instead of skipgram padding



ETAPA C: MODELADO DE INTERACCIÓN

5. Arquitectura de Datos

5.1 Esquema del Modelo Relacional

El modelo relacional implementa un esquema híbrido transaccional-analítico. Las tablas transaccionales (sales, sale_details, products) están diseñadas para soportar operaciones de escritura de alta frecuencia, mientras que los índices sobre created_at y las claves foráneas permiten consultas agregadas eficientes para el Dashboard DSS sin degradar el rendimiento operativo.

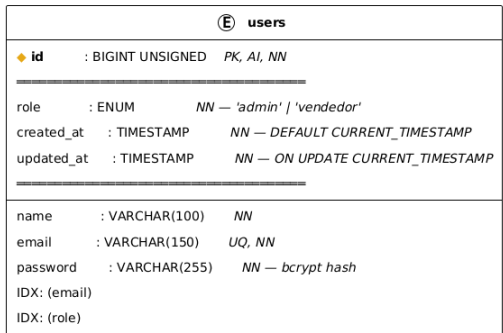
Tablas del modelo:

- users: Autenticación y RBAC. Controla el acceso al módulo analítico.
- products: Catálogo de repuestos. Almacena cost (costo de adquisición) como métrica base del margen de contribución.
- sales: Encabezado transaccional. Registra invoice_status e invoice_xml_hash para trazabilidad fiscal ante el SIN.
- sale_details: Detalle de línea. Congela unit_price y unit_cost al momento de la transacción para garantizar la consistencia histórica del análisis de rentabilidad.

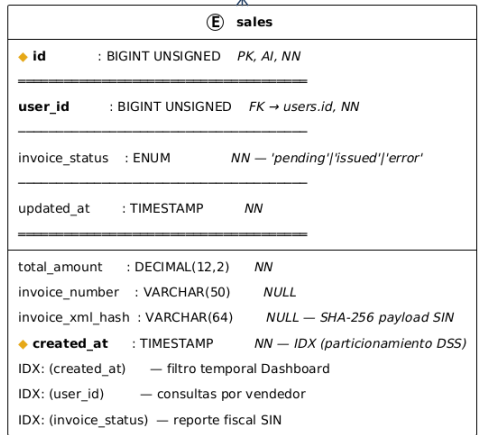
Modelo Relacional — Sistema DSS de Repuestos Automotrices
Esquema optimizado para consultas transaccionales y analíticas (HTAP)

Símbolo	Significado
PK	Clave primaria
FK	Clave foránea
UQ	Restricción UNIQUE
IDX	Índice para consultas analíticas
NN	NOT NULL
AI	AUTO_INCREMENT
◆	Campo crítico del motor DSS

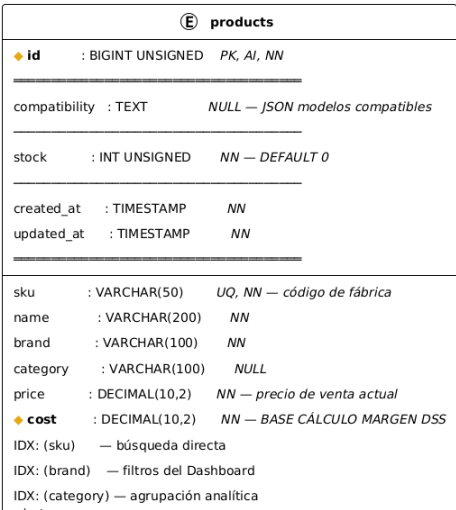
Please use CSS style instead of skinparam padding



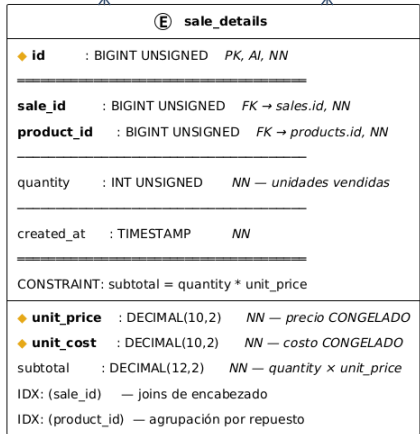
1 usuario
registra 0..* ventas



1 venta
compone 1..* detalles



1 producto
aparece en 0..* detalles



5.2 Diccionario de Datos

El diccionario de datos describe en detalle cada tabla del modelo relacional: columnas, tipos de dato, constraints y propósito analítico de cada campo.

Tabla: users

Entidad de autenticación y control de acceso basado en roles (RBAC).

Columna	Tipo	Constraints	Descripción
id	BIGINT UNSIGNED	PK	AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL	Nombre completo del usuario
email	VARCHAR(150)	UNIQUE NOT NULL	Correo electrónico de acceso
password	VARCHAR(255)	NOT NULL	Hash bcrypt
role	ENUM('admin','vendedor')	NOT NULL DEFAULT 'vendedor'	Rol que determina acceso al DSS
created_at	TIMESTAMP	NOT NULL	Fecha de creación del registro
updated_at	TIMESTAMP	NOT NULL	Última modificación

Tabla: products

Catálogo de repuestos automotrices. Contiene precio de venta y costo de adquisición como base del cálculo de margen.

Columna	Tipo	Constraints	Descripción
id	BIGINT UNSIGNED	PK	AUTO_INCREMENT
sku	VARCHAR(50)	UNIQUE NOT NULL	Código de fábrica del repuesto
name	VARCHAR(200)	NOT NULL	Nombre descriptivo del repuesto
brand	VARCHAR(100)	NOT NULL	Marca del repuesto

Columna	Tipo	Constraints	Descripción
compatibility	TEXT	NULL	Modelos de vehículo compatibles (JSON o texto libre)
price	DECIMAL(10,2)	NOT NULL	Precio de venta actual
cost	DECIMAL(10,2)	NOT NULL	Costo de adquisición (base del margen de contribución)
stock	INT UNSIGNED	NOT NULL DEFAULT 0	Unidades disponibles en almacén
category	VARCHAR(100)	NULL	Categoría (frenos, filtros, bujías, etc.)
created_at	TIMESTAMP	NOT NULL	
updated_at	TIMESTAMP	NOT NULL	

Tabla: sales

Encabezado transaccional de cada venta. Indexado por created_at para particionamiento temporal en el Dashboard.

Columna	Tipo	Constraints	Descripción
id	BIGINT UNSIGNED	PK	AUTO_INCREMENT
user_id	BIGINT UNSIGNED	FK → users.id NOT NULL	Vendedor que registró la venta
total_amount	DECIMAL(12,2)	NOT NULL	Monto total de la venta
invoice_status	ENUM('pending','issued','error')	NOT NULL DEFAULT 'pending'	Estado de la factura electrónica ante el SIN
invoice_xml_hash	VARCHAR(64)	NULL	Hash SHA-256 del XML enviado al SIN para verificación
invoice_number	VARCHAR(50)	NULL	Número de factura emitido

Columna	Tipo	Constraints	Descripción
created_at	TIMESTAMP	NOT NULL INDEX	Fecha de venta (clave de particionamiento analítico)
updated_at	TIMESTAMP	NOT NULL	

Tabla: sale_details

Detalle de granularidad fina por línea de venta. Preserva el precio histórico congelado al momento de la transacción.

Columna	Tipo	Constraints	Descripción
id	BIGINT UNSIGNED	PK	AUTO_INCREMENT
sale_id	BIGINT UNSIGNED	FK → sales.id NOT NULL	Venta a la que pertenece
product_id	BIGINT UNSIGNED	FK → products.id NOT NULL	Repuesto vendido
quantity	INT UNSIGNED	NOT NULL	Unidades vendidas
unit_price	DECIMAL(10,2)	NOT NULL	Precio de venta congelado al momento de la transacción
unit_cost	DECIMAL(10,2)	NOT NULL	Costo de adquisición congelado (para margen histórico correcto)
subtotal	DECIMAL(12,2)	NOT NULL	quantity * unit_price
created_at	TIMESTAMP	NOT NULL	

Índices adicionales recomendados para consultas analíticas

Tabla	Columna(s)	Tipo	Justificación analítica
sales	created_at	INDEX	Particionamiento temporal para filtros por rango de fechas en el Dashboard
sales	invoice_status	INDEX	Filtrado rápido de facturas por estado para reportes fiscales
sale_details	product_id	INDEX (FK)	Joins frecuentes para calcular rotación por producto
products	sku	UNIQUE INDEX	Búsqueda directa de repuesto por código de fábrica

ETAPA D: AGILE READINESS

Esta etapa traduce el diagnóstico (Etapa A) y el diseño técnico (Etapas B y C) en unidades de trabajo ejecutables por el Squad. El Product Backlog se redacta bajo el estándar INVEST y cada Historia de Usuario será registrada como un Issue independiente en GitHub Projects (vista Board / Kanban), vinculado al repositorio del proyecto, conforme a lo establecido en la estrategia de preparación ágil.

6. Product Backlog

El backlog se organiza en cuatro épicas que se derivan directamente de las entidades y servicios definidos en el modelo de clases y el modelo relacional (User, Product, Sale, SaleDetail, DSSAnalyzer): Seguridad y Acceso, Gestión de Catálogo, Ventas y Facturación Electrónica, y Motor DSS / Dashboard Analítico. Cada Historia de Usuario cumple los seis atributos INVEST: Independiente, Negociable, Valiosa, Estimable, Pequeña (Small) y Testeable.

Épica 1 — Seguridad y Acceso (RBAC)

ID / Título	HU-01 — Registro y autenticación de usuarios
Historia de Usuario	Como administrador del sistema, quiero registrar usuarios del negocio con un rol asignado (admin o vendedor),

		para controlar quién puede acceder al módulo operativo y quién puede acceder al módulo analítico DSS.
Criterios Aceptación (Definition Ready)	de of	☐ El formulario de registro exige name, email (único) y password (hash bcrypt), conforme a la tabla users. ☐ El campo role solo acepta los valores 'admin' o 'vendedor', con 'vendedor' como valor por defecto. ☐ Un correo duplicado es rechazado con un mensaje de error claro (constraint UNIQUE). ☐ Las contraseñas nunca se almacenan ni se transmiten en texto plano.
Estimación (Story Points)		3
Prioridad (MoSCoW)		Must have

ID / Título		HU-02 — Inicio de sesión con token JWT
Historia Usuario	de	Como usuario registrado (admin o vendedor), quiero iniciar sesión con mi correo y contraseña, para obtener un token de acceso (JWT) que me identifique en cada solicitud a la API.
Criterios Aceptación (Definition Ready)	de of	☐ Credenciales válidas devuelven un token JWT y los datos básicos del usuario (id, name, role). ☐ Credenciales inválidas devuelven HTTP 401 sin revelar si el error es el correo o la contraseña. ☐ El token expira según la configuración de Laravel Sanctum y debe renovarse al expirar. ☐ El token se almacena en el cliente (LocalStorage) y se envía como header Authorization: Bearer.
Estimación (Story Points)		2
Prioridad (MoSCoW)		Must have

ID / Título		HU-03 — Restricción de acceso al módulo DSS por rol
Historia Usuario	de	Como administrador del negocio,

		quiero que solo los usuarios con rol 'admin' puedan ingresar al Dashboard analítico, para evitar que el personal de ventas acceda a información financiera sensible (márgenes y costos).
Criterios Aceptación (Definition Ready)	de of	☐ El middleware AuthMiddleware valida el JWT y ejecuta assertRole(user, 'admin') antes de delegar al DashboardController. ☐ Un usuario con rol 'vendedor' que solicite /api/dashboard recibe HTTP 403 Forbidden. ☐ Un token inválido o expirado recibe HTTP 401 Unauthorized y el frontend redirige a /login. ☐ El método hasAccessToDSS() de la entidad User refleja correctamente la regla de negocio anterior.
Estimación (Story Points)		3
Prioridad (MoSCoW)		Must have

Épica 2 — Gestión de Catálogo de Repuestos

ID / Título		HU-04 — Registro de productos con costo y precio
Historia Usuario	de	Como vendedor o administrador, quiero registrar un repuesto con su SKU, marca, compatibilidad, precio de venta y costo de adquisición, para que el sistema disponga de la información financiera base para calcular el margen de contribución.
Criterios Aceptación (Definition Ready)	de of	☐ El campo sku es único y obligatorio (constraint UNIQUE NOT NULL sobre la tabla products). ☐ Los campos price y cost se almacenan como DECIMAL(10,2) y son obligatorios. ☐ El campo stock no permite valores negativos (constraint >= 0). ☐ El método getContributionMargin() retorna price - cost de forma consistente con el dato persistido.
Estimación (Story Points)		3
Prioridad (MoSCoW)		Must have

ID / Título		HU-05 — Edición y baja de productos del catálogo
Historia Usuario	de	Como administrador del negocio, quiero actualizar el precio, costo o stock de un repuesto, o darlo de baja del catálogo activo, para mantener el catálogo alineado con la realidad del inventario y los proveedores.
Criterios Aceptación (Definition Ready)	de of	☐ La edición de price o cost no modifica los registros históricos en sale_details (unit_price y unit_cost congelados). ☐ El sistema valida que sku siga siendo único al editar. ☐ Un producto dado de baja no aparece en nuevas ventas, pero su histórico se conserva intacto.
Estimación (Story Points)		2
Prioridad (MoSCoW)		Should have

ID / Título		HU-06 — Búsqueda y filtrado de productos
Historia Usuario	de	Como vendedor, quiero buscar un repuesto por SKU, nombre, marca o categoría, para registrar una venta rápidamente sin recorrer manualmente todo el catálogo.
Criterios Aceptación (Definition Ready)	de of	☐ La búsqueda por sku retorna resultados exactos en menos de 300 ms (apoyada en el índice IDX: sku). ☐ El filtrado por brand y category utiliza los índices definidos en el modelo relacional. ☐ La respuesta de la API incluye stock disponible para evitar vender productos agotados.
Estimación (Story Points)		2
Prioridad (MoSCoW)		Could have

ID / Título		HU-07 — Registro de una venta con múltiples detalles
Historia Usuario	de	Como vendedor, quiero registrar una venta compuesta de uno o más repuestos con su cantidad, para que el sistema calcule el monto total y conserve el precio y costo histórico de cada línea.
Criterios Aceptación (Definition Ready)	de of	☐ Cada SaleDetail almacena quantity, unit_price y unit_cost congelados al momento exacto de la transacción. ☐ El campo subtotal se calcula como quantity × unit_price y se valida contra ese constraint. ☐ Sale.total_amount equivale a la suma de los subtotal de sus SaleDetail asociados. ☐ Una venta sin al menos un SaleDetail es rechazada (composición fuerte 1..* definida en el diagrama de clases).
Estimación (Story Points)		5
Prioridad (MoSCoW)		Must have

ID / Título		HU-08 — Emisión de factura electrónica (SIN)
Historia Usuario	de	Como vendedor, quiero emitir la factura electrónica asociada a una venta registrada, para cumplir con la normativa fiscal vigente y dejar trazabilidad verificable de cada transacción.
Criterios Aceptación (Definition Ready)	de of	☐ El método issueInvoice() actualiza invoice_status a 'issued' y genera invoice_number e invoice_xml_hash (SHA-256). ☐ Si el servicio de facturación falla, invoice_status pasa a 'error' y la venta no queda en estado ambiguo. ☐ El método verifyInvoiceHash() permite confirmar la integridad del XML enviado al SIN. ☐ El estado por defecto de toda venta nueva es 'pending' hasta que se confirme la emisión.
Estimación (Story Points)		5

Prioridad (MoSCoW)	Must have
---------------------------	-----------

ID / Título	HU-09 — Consolidado de estado de facturación	
Historia Usuario	de	Como administrador del negocio, quiero ver cuántas ventas están en estado 'issued', 'pending' o 'error' en un rango de fechas, para detectar a tiempo fallas en el proceso de facturación electrónica antes de que generen incumplimiento fiscal.
Criterios Aceptación (Definition Ready)	de of	☐ El endpoint getInvoiceSummary() agrupa por invoice_status usando el índice IDX (invoice_status). ☐ El resultado se expone en el Dashboard como un gráfico de dona (issued / pending / error). ☐ El rango de fechas por defecto es de los últimos 30 días, igual que el resto de los KPIs del Dashboard.
Estimación (Story Points)	3	
Prioridad (MoSCoW)	Must have	

Épica 4 — Motor DSS y Dashboard Analítico

ID / Título	HU-10 — Cálculo de tasa de rotación y margen por producto	
Historia Usuario	de	Como administrador del negocio, quiero que el sistema calcule automáticamente la tasa de rotación y el margen de ganancia neta de cada repuesto, para tomar decisiones de compra y precio basadas en datos reales y no en la intuición del personal.
Criterios Aceptación (Definition Ready)	de of	☐ $\text{computeRotationRate}(p) = \text{total_sold} / \text{período_días}$, sobre el rango analizado por DSSAnalyzer. ☐ $\text{computeMarginRate}(p) = (\text{total_revenue} - \text{total_cost}) / \text{total_revenue}$, evitando división por cero cuando $\text{total_revenue} = 0$. ☐ Los umbrales rotationThreshold (0.5) y marginThreshold (0.20) son configurables, no están escritos en duro en el código.

	☐ El cálculo se basa en la QUERY 3 del diagrama de secuencia (agregación de sale_details y sales por producto).
Estimación (Story Points)	5
Prioridad (MoSCoW)	Must have

ID / Título		HU-11 — Clasificación matricial Estrella / Hueso
Historia Usuario	de	Como administrador del negocio, quiero ver el catálogo clasificado en los cuadrantes Estrella, Interrogante, Vaca y Hueso, para identificar qué productos inmovilizan capital de trabajo y cuáles debo priorizar para reposición.
Criterios Aceptación (Definition Ready)	de of	☐ classifyProduct() asigna ESTRELLA cuando rotación $\geq \theta$ y margen $\geq \theta$, y HUESO cuando ambas son menores al umbral. ☐ buildStarHusoMatrix() retorna los cuatro cuadrantes y permite extraer top_star[5] y critical_huso[5]. ☐ La matriz se renderiza en <StarHusoMatrix> como un scatter plot con ejes rotación $\times$ margen y color por cuadrante. ☐ El 100% del catálogo de productos activos queda clasificado en alguno de los cuatro cuadrantes, según el objetivo SMART del proyecto.
Estimación (Story Points)		8
Prioridad (MoSCoW)		Must have

ID / Título		HU-12 — Dashboard interactivo con KPIs generales
Historia Usuario	de	Como administrador del negocio, quiero ver en un solo panel los ingresos totales, número de ventas y productos activos del período, para tener visibilidad financiera inmediata sin esperar reportes manuales tardíos.
Criterios Aceptación (Definition Ready)	de of	☐ GET /api/dashboard responde con kpis, matrix, top_star, critical_huso e invoice_summary en un único payload JSON. ☐ El componente <KPICards> consume kpis.total_revenue, kpis.total_sales y kpis.active_products.

	☐ El filtro de fecha por defecto es de los últimos 30 días y es ajustable por el usuario administrador. ☐ El tiempo de generación del Dashboard se mide como métrica de éxito del proyecto (Canvas MVP, Etapa A).
Estimación (Story Points)	5
Prioridad (MoSCoW)	Must have

6.1 Resumen del Backlog y Tablero Kanban

La siguiente tabla resume el backlog priorizado bajo la heurística MoSCoW (Must / Should / Could have). Cada fila corresponde a un Issue independiente en GitHub Projects, vinculado a la columna del tablero Kanban (Backlog → To Do → In Progress → In Review → Done).

ID	Historia de Usuario	Épica	Story Points	Prioridad
HU-01	Registro y autenticación de usuarios	Seguridad y Acceso	3	Must have
HU-02	Inicio de sesión con token JWT	Seguridad y Acceso	2	Must have
HU-03	Restricción de acceso al DSS por rol	Seguridad y Acceso	3	Must have
HU-04	Registro de productos con costo y precio	Catálogo	3	Must have
HU-05	Edición y baja de productos	Catálogo	2	Should have
HU-06	Búsqueda y filtrado de productos	Catálogo	2	Could have
HU-07	Registro de venta con múltiples detalles	Ventas y Facturación	5	Must have
HU-08	Emisión de factura electrónica (SIN)	Ventas y Facturación	5	Must have
HU-09	Consolidado de estado de facturación	Ventas y Facturación	3	Must have
HU-10	Cálculo de rotación y margen por producto	Motor DSS / Dashboard	5	Must have
HU-11	Clasificación matricial Estrella / Hueso	Motor DSS / Dashboard	8	Must have

ID	Historia de Usuario	Épica	Story Points	Prioridad
HU-12	Dashboard interactivo con KPIs generales	Motor DSS / Dashboard	5	Must have

Total estimado del backlog: 46 Story Points distribuidos en 12 Historias de Usuario. El tablero Kanban de GitHub Projects se vinculará directamente al repositorio mi-sistema-ventas-dss y su evidencia visual (captura de pantalla) se incorporará en la Sección 7 (Plan de Ejecución) del entregable final.

7. Definition of Ready (DoR)

La Definition of Ready establece las condiciones mínimas que debe cumplir el proyecto — a nivel de entorno, repositorio, base de datos y backlog — antes de que el Squad inicie la codificación. Su propósito es eliminar ambigüedades de arranque: el lunes de inicio del sprint, ningún desarrollador debería tener una pregunta sin respuesta sobre qué construir, dónde y con qué estructura de datos.

7.1 Checklist General de Preparación

Entorno y Repositorio:

- ☐ El repositorio `mi-sistema-ventas-dss` existe en GitHub con una instalación limpia de Laravel.
- ☐ La estructura de carpetas `/app`, `/database`, `/docs`, `/resources` y `/routes` está creada conforme a la Sección 2 de este documento.
- ☐ El archivo `.gitignore` excluye `.env`, `/vendor` y `/node_modules`.
- ☐ El `README.md` incluye los miembros del Squad y el enlace al PDF consolidado del entregable.
- ☐ Cada integrante del Squad tiene acceso de escritura al repositorio y a GitHub Projects.

Base de Datos:

- ☐ Las migraciones de `users`, `products`, `sales` y `sale_details` están versionadas en `/database/migrations`.
- ☐ Los índices analíticos (`created_at`, `invoice_status`, `sku`, `brand`, `category`) están declarados en las migraciones, no se agregarán "después".
- ☐ Existen seeders que poblarán datos simulados suficientes para que el motor DSS clasifique al menos un producto en cada cuadrante (Estrella, Interrogante, Vaca, Hueso).
- ☐ El constraint `subtotal = quantity × unit_price` está validado a nivel de aplicación (Service Layer), no solo de documentación.

Backlog y Tablero Ágil:

- ☐ Las 12 Historias de Usuario están registradas como Issues independientes en GitHub Projects.
- ☐ Cada Issue está vinculado a la columna correspondiente del tablero Kanban (Backlog / To Do / In Progress / In Review / Done).
- ☐ Cada Issue contiene su Historia de Usuario completa (Como / Quiero / Para) y su checklist de Criterios de Aceptación en Markdown.
- ☐ Las dependencias entre Historias están señaladas (por ejemplo: HU-07 debe completarse antes de HU-08, y HU-04 antes de HU-10).

7.2 Checklist Técnico por Componente

Componente	Condición de Listo (Ready)
Modelos Eloquent	☐ User, Product, Sale y SaleDetail están creados con sus relaciones (hasMany / belongsTo) reflejando el diagrama de clases.

Componente	Condición de Listo (Ready)
	☐ Los campos price, cost, unit_price y unit_cost están tipados como decimal:10,2 en los \$casts del modelo.
Migraciones	☐ Toda migración es reversible (down() implementado). ☐ Las claves foráneas (user_id, sale_id, product_id) tienen su constraint declarado explícitamente.
DSSAnalyzer (Service)	☐ La clase existe en app/Services con los umbrales rotationThreshold y marginThreshold como propiedades configurables. ☐ Las firmas de los métodos (classifyProduct, buildStarHusoMatrix, getDashboardKPIs) coinciden con el diagrama de clases.
Dashboard Controller	☐ Las rutas /api/dashboard, /api/dashboard/matrix y /api/dashboard/kpis están registradas en routes/api.php. ☐ El middleware auth:sanctum y la verificación de rol 'admin' están aplicados antes de delegar al servicio.
Frontend (React)	☐ El componente DashboardPage y el cliente AxiosClient están scaffoldados en resources/js. ☐ Las librerías de gráficos (ApexCharts o equivalente) están instaladas y referenciadas en package.json.
Facturación Electrónica	☐ El contrato del servicio externo de facturación (mock o real) está definido: qué recibe y qué retorna issueInvoice(). ☐ El criterio para marcar una venta como invoice_status = 'error' está acordado por el Squad.

7.3 Criterio de Cierre de la Etapa

Se considera que el proyecto cumple la Definition of Ready cuando el 100% de los ítems de las listas de verificación anteriores está marcado, el tablero Kanban es visible públicamente para el equipo docente, y al menos una Historia de Usuario (HU-01) puede moverse de la columna "To Do" a "In Progress" sin requerir información adicional. A partir de ese punto, el Squad puede iniciar la codificación el día lunes